

Session Overview



- Yesterday , BLINK LED
- Today
- Config POT to read values linke speed , SOC , break and temp and print in on terminal

STM32 Workshop

Day 2 – ADC on STM32

Analog-to-Digital Conversion for EV Dashboard
Accelerator · Brake · SOC · Motor Temperature

ADC Architecture

12-bit Resolution

CubeMX Config

UART Display

Session Overview

What you'll learn today

01 ADC Architecture

How the STM32 ADC converts analog signals to digital values

02 12-bit Resolution & Mapping

Voltage range 0–3.3V → 0–4095, formula and examples

03 CubeMX Configuration

Enable ADC1, channels CH0–CH3 on PA0–PA3

04 Reading 4 Sensors

Sequential polling with channel switching — one channel at a time

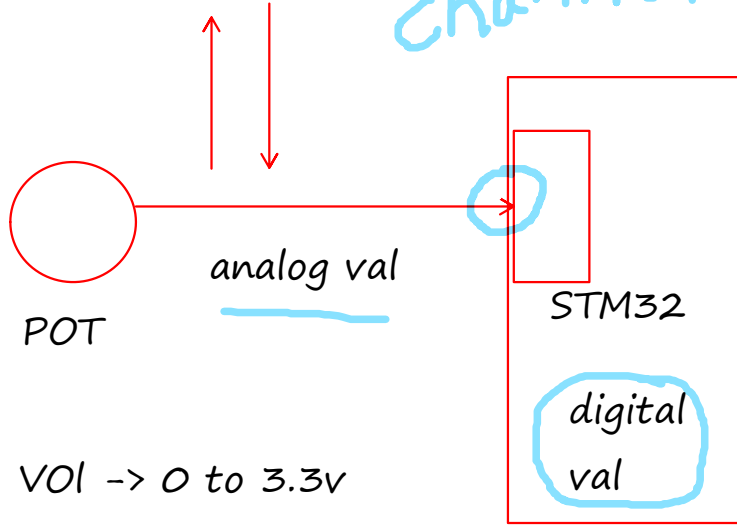
05 PICSimLab Potentiometers

Use virtual potentiometers as analog inputs

06 UART Display at 115200

Print all 4 ADC readings over serial with HAL_UART_Transmit

channel

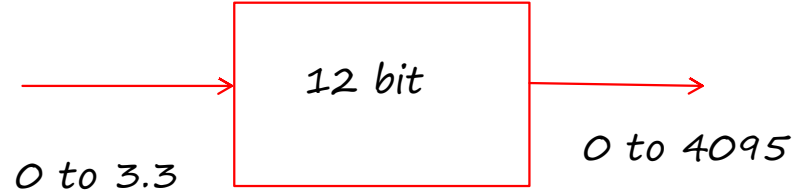


1.3 2.3 2.5

18

ADC

8 bit $\rightarrow$ 0 to 255
10 bit $\rightarrow$ 0 to 1023
12 bit $\rightarrow$ 0 to 4096
ADC



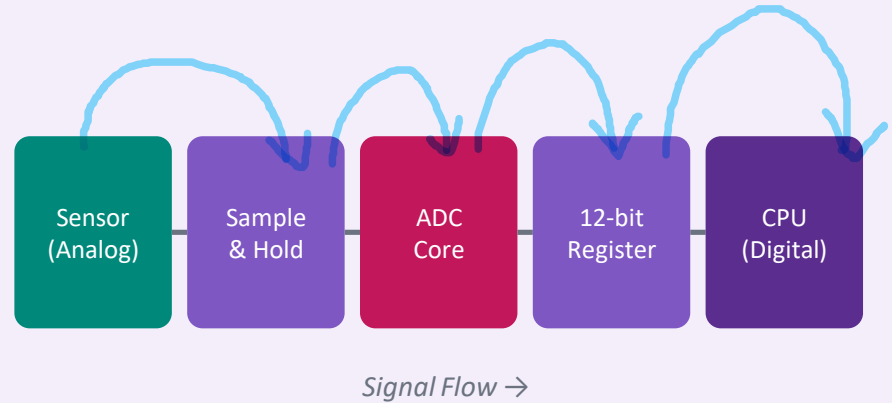
resolution
width ADC results

What is an ADC?

Analog-to-Digital Converter — the bridge between real world and MCU

The Real World is Analog

- Voltage, temperature, pressure — continuous values
- STM32 CPU is digital — it only understands 0s and 1s
- ADC samples the analog signal at regular intervals
- Converts to a discrete binary number (0 to 4095 for 12-bit)



STM32F103 ADC Facts

- Resolution: 12-bit (0 – 4095)
- Vref: 0 – 3.3 V
- Up to 18 channels | Single / Scan / Continuous modes



Key Takeaway: ADC is how the MCU 'hears' the physical world — it digitizes sensor voltages for firmware processing.

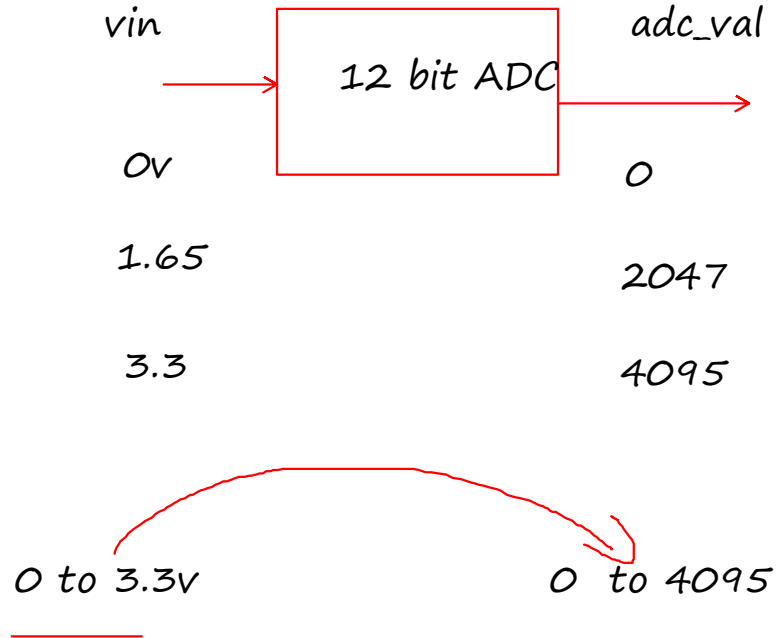
0 % 0 v to 0

50% 1.65 to 2047

100 3.3v to 4095

$$\begin{aligned} \text{adc_val} &= V/v_{\text{ref}} * 4095 \\ &= 1.65/3.3 * 4095 \\ &= 2047 \end{aligned}$$

$$\begin{aligned} v_{\text{in}} &= \text{adc_val} * v_{\text{ref}} / 4095 \\ &= 2047 * 3.3 / 4095 \end{aligned}$$



12-bit Resolution & Voltage Mapping

Understanding the conversion formula

$$\text{ADC Value} = (V_{in} / V_{ref}) \times (2^{12} - 1)$$

$$= (V_{in} / 3.3) \times 4095$$

$V_{in} = 0.0 \text{ V}$

ADC = 0

0% · Pedal fully released

$V_{in} = 1.65 \text{ V}$

ADC = 2047

50% · Mid-range

$V_{in} = 3.3 \text{ V}$

ADC = 4095

100% · Pedal fully pressed

Reverse: Voltage = $\text{ADC_Value} \times 3.3 / 4095$ e.g. ADC=2048 → 1.65 V

Key Takeaway: Every 1-unit ADC step = $3.3\text{V} / 4095 \approx 0.806 \text{ mV}$. Resolution = 12-bit means 4096 discrete levels.

CubeMX: Configuring ADC1

Single-channel polling — enable IN0–IN3 on PA0–PA3

1 Enable ADC1 Channels

Pinout & Config → ADC1 → Enable IN0 (PA0), IN1 (PA1), IN2 (PA2), IN3 (PA3) as Analog input

2 Keep Default Mode (Single)

Parameter Settings → Scan Conversion: Disabled | Continuous: Disabled | # Conversions: 1

3 No DMA Required

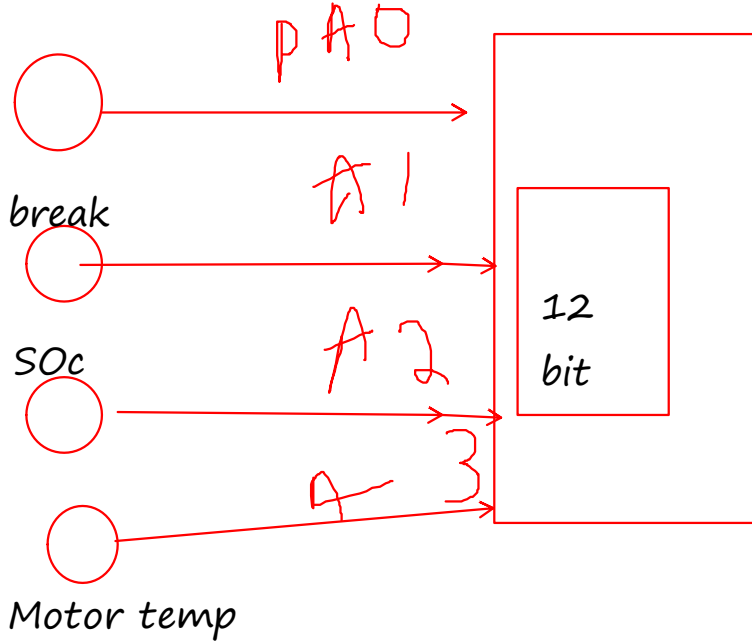
PICSimLab reads one channel at a time. Select the channel in firmware before each HAL_ADC_Start() call.

Channel → Pin → Signal

Channel	Pin	Signal
IN0	PA0 ✓	<u>Accelerator Pedal</u>
IN1	PA1 ✓	Brake Pedal
IN2	PA2 ✓	<u>Battery SOC</u>
IN3	PA3 ✓	<u>Motor Temp</u>

⚠ **Common Mistake:** PICSimLab ADC does NOT support Scan/DMA multi-channel — read one channel at a time by switching channels in firmware.

acc



0 to 4095 0 to 100

$$\text{adc_val} * 100 / 4095 \quad 48$$
$$2000 * 100 / 4095$$

reading the values

`acc = read_adc(channel 0)`

scale 0 to 100%

0 to 100%

0 to 100%

0 to 150

Reading ADC Values — HAL Polling Mode

One channel at a time: the correct approach for PICSimLab

```
// Helper: read a specific ADC channel
uint32_t ADC_Read(uint32_t channel) {
    ADC_ChannelConfTypeDef cfg = {0};
    cfg.Channel = channel;
    cfg.Rank      = ADC_REGULAR_RANK_1;
    cfg.SamplingTime = ADC_SAMPLETIME_55CYCLES_5;
    HAL_ADC_ConfigChannel(&hadc1, &cfg);

    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 100);
    uint32_t val = HAL_ADC_GetValue(&hadc1);
    HAL_ADC_Stop(&hadc1);
    return val;
}
```

ADC_ChannelConfTypeDef

Selects which input pin
to sample before each read.



HAL_ADC_ConfigChannel()

Reconfigures ADC to the
chosen channel at runtime.

HAL_ADC_GetValue()

Returns 12-bit result
(0 – 4095) after conversion.



Key Takeaway: Switching channels via HAL_ADC_ConfigChannel() is the correct way to read multiple pins one at a time.

Reading All 4 Channels — Sequential Polling



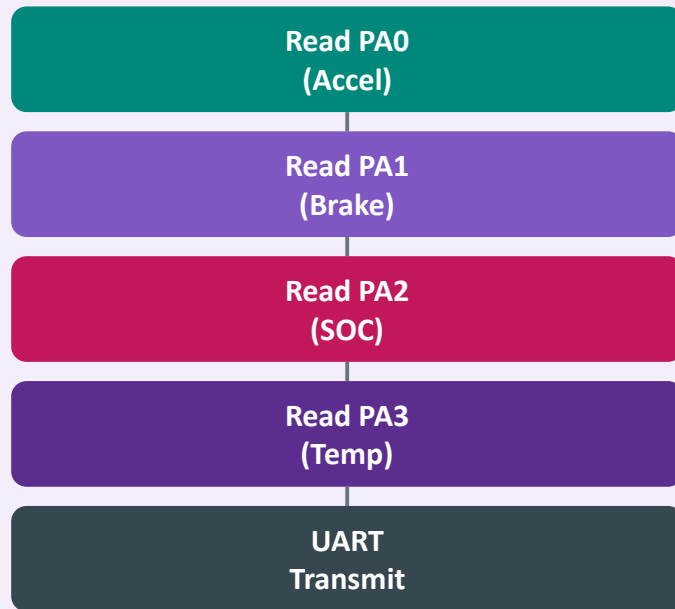
Call `ADC_Read()` for each channel in order, then transmit

```
// In your main loop (using the helper from previous
slide):
uint32_t accel = ADC_Read(ADC_CHANNEL_0); // PA0
uint32_t brake = ADC_Read(ADC_CHANNEL_1); // PA1
uint32_t soc   = ADC_Read(ADC_CHANNEL_2); // PA2
uint32_t temp  = ADC_Read(ADC_CHANNEL_3); // PA3

// Scale to useful units:
uint8_t accel_pct = accel * 100 / 4095;
uint8_t brake_pct = brake * 100 / 4095;
uint8_t soc_pct   = soc   * 100 / 4095;
uint8_t temp_c    = temp  * 150 / 4095; // 0-150°C

HAL_Delay(100); // 10 Hz update rate
```

Execution Order



⚠ Common Mistake: Do NOT call `HAL_ADC_Start()` without first calling `HAL_ADC_ConfigChannel()` — you'll keep reading the same channel.

PICSimLab: Potentiometers as Analog Inputs

Simulating real-world sensor voltages on PA0–PA3

Wiring Potentiometers in PICSimLab

1

Potentiometer 1 → PA0

Accelerator Pedal (0–100%)

2

Potentiometer 2 → PA1

Brake Pedal (0–100%)

3

Potentiometer 3 → PA2

Battery SOC simulation

4

Potentiometer 4 → PA3

Motor Temperature (0–150°C)

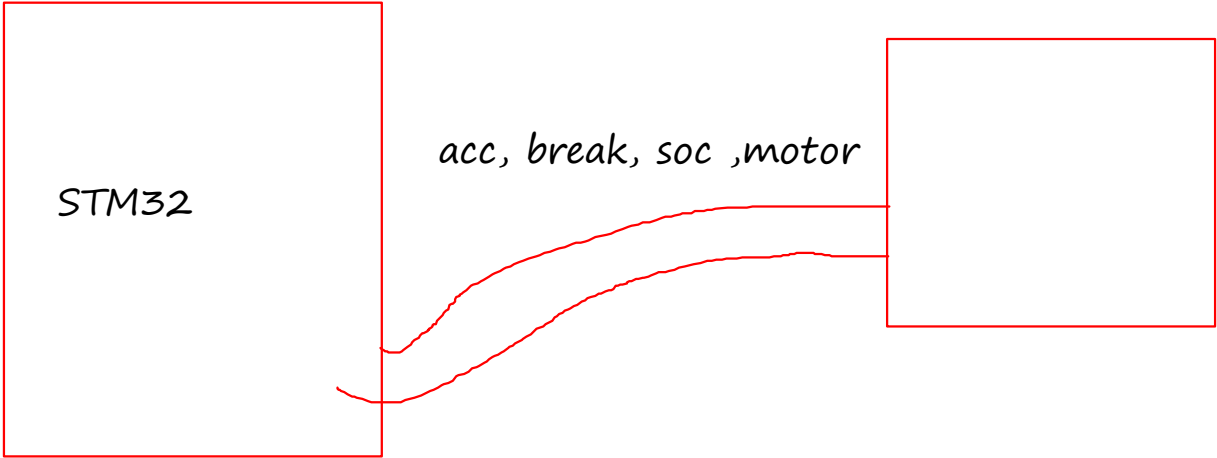
Simulation Tips

-  Add a potentiometer component per analog signal
-  Connect each pot wiper to its GPIO pin (PA0–PA3)
-  Slide the pot to vary the input voltage (0–3.3 V)
-  Firmware reads each channel separately, one by one
-  PICSimLab does NOT support Scan/DMA — polling only
-  Check UART terminal to verify all 4 values update

 **Try It Yourself:** Add 4 potentiometers in PICSimLab (one per pin PA0–PA3). Use `ADC_Read()` with channel switching to read each one.

print V terminal

I/O vertm



Displaying ADC Readings via UART

115200 baud | HAL_UART_Transmit | sprintf formatting

```
#include <stdio.h> // for sprintf
char buf[128];
uint32_t adc_buf[4]; // filled by DMA

// Scale to percentage
uint8_t accel = adc_buf[0] * 100 / 4095;
uint8_t brake = adc_buf[1] * 100 / 4095;
uint8_t soc   = adc_buf[2] * 100 / 4095;
// Temp: 0-3.3V maps to 0-150°C
uint8_t temp  = adc_buf[3] * 150 / 4095;

int len = sprintf(buf,
    "ACCEL=%3d%% BRAKE=%3d%% SOC=%3d%% TEMP=%3d°C\r\n",
    accel, brake, soc, temp);

HAL_UART_Transmit(&huart1, (uint8_t*)buf, len, 100);
```

UART1 Config Reminder

```
Baud Rate : 115200
Word Length: 8 bits
Stop Bits : 1
Parity : None
Flow Ctrl : None
```

Expected UART Output

```
ACCEL= 45% BRAKE= 0%
SOC= 78% TEMP= 65C
ACCEL=100% BRAKE= 0%
SOC= 70% TEMP= 72C
```

⚠ **Common Mistake:** Don't use `printf()` directly — it requires retargeting. Use `sprintf()` + `HAL_UART_Transmit()` instead.

-> read val from POT1 2 3 4 -> PA0 PA1 PA2 PA3 of STM32
and print it on i/o virtual terminal using uart

ADC

STM32cubeIDE -> ADC pin
UART

generate code -> initial

main() while(1) -> read adc val channel 0 1 2 3 -> (0 to 4095) read_adc(channel number)
-> scale it to 0 to 100
-> print it on I/O virtual terminal

Connecting to the EV Dashboard



How today's ADC readings feed the full EV system

PA0 · CH0 — Accelerator

Dashboard Role:
Motor torque command
(0–100%)

PA1 · CH1 — Brake Pedal

Dashboard Role:
Regen braking + fault check
(if ACCEL & BRAKE both >80%)

PA2 · CH2 — Battery SOC

Dashboard Role:
State of Charge display
Python dashboard gauge

PA3 · CH3 — Motor Temp

Dashboard Role:
Thermal protection trigger
(FAULT if Temp > threshold)



Key Takeaway: Today's 4-channel ADC reader IS the sensor input layer of your full EV Dashboard capstone project.

Session Summary & What's Next

- ✓ Understood ADC architecture and 12-bit resolution
- ✓ Configured ADC1 (CH0–CH3, PA0–PA3) in CubeMX
- ✓ Read 4 analog signals using HAL polling and DMA
- ✓ Used PICSimLab potentiometers as analog inputs
- ✓ Transmitted formatted ADC data over UART at 115200

Coming Up — Day 3

- UART Command Shell
- Fault Detection Logic
- EV State Machine
- Python Dashboard Integration

 Practice: Wire 4 pots in PICSimLab, configure ADC1+DMA, print readings over UART before next session!